

Topology Statistics Common Foundation Specification

Normative TS0 API for Exact Distributions, Occupancies, Axes, and Shared Serialization in `mdstats`

`mdstats`

2026-07-14 (implemented TS0 revision)

Contents

Purpose and status	2
Motivation	3
Responsibility boundaries	3
Owned by <code>_common.py</code>	3
Not owned by <code>_common.py</code>	3
Module layout and import policy	3
Public constants	4
Exception hierarchy	4
Mathematical conventions	4
Population summary	4
Exact discrete distribution	4
Catalog occupancy	4
Data structures	5
ScalarSummary	5
Invariants	5
Serialization	5
DiscreteCountDistribution	5
Meaning	5
Properties and queries	6
Invariants	6
StateFrameGroup	6
CatalogOccupancyStatistics	6
Dense state contract	6
Trajectory versus ensemble behavior	7
Properties	7
FrameAxis	7
Meaning	7
Trajectory constraints	7
Ensemble constraints	7
Properties	7
ScalarSeries	8
Invariants	8
Public functions	8
<code>compute_scalar_summary</code>	8
Input constraints	8
Algorithm	8
<code>compute_discrete_count_distribution</code>	8
Input constraints	8
Algorithm	9
<code>compute_catalog_occupancy_statistics</code>	9
Input constraints	9

Algorithm	9
build_frame_axis	9
build_scalar_series	9
expand_state_values_to_frames	10
Canonical JSON and payload digest	10
Serialization contract	10
Immutability and defensive copying	11
Trajectory and ensemble semantics	11
Edge cases and warnings	11
Constant series	11
Multiple modes	11
Unobserved declared states	11
Correlated trajectory frames	11
Large state expansions	11
Floating-point quantiles	12
Time duration	12
Usage examples	12
Exact constant contact distribution	12
Recurrent trajectory-state occupancy	12
Immutable time-aligned count series	12
Testing requirements	12
Implementation notes for later stages	13
TS1 atomic statistics - implemented	13
TS2 framework statistics - implemented	13
TS3 temporal statistics - implemented	13
Accepted TS0 decisions	13
Final TS0 contract	14

Purpose and status

This document specifies the implemented private module

`mdstats/analysis/topology_statistics/_common.py`

and its public re-export surface through

```
mdstats.analysis.topology_statistics
mdstats.analysis
mdstats
```

for topology-statistics Stage TS0. The implementation was introduced in `mdstats 0.17.0a0` and is the unchanged foundation used by TS1 through TS4 in `0.17.0a1-0.17.0a4`.

TS0 provides graph-independent statistical primitives used later by atomic, framework, temporal, and combined topology-statistics modules. It does not know what an atomic edge, framework edge, bridge signature, ring, site, or cage means.

The central boundary is

catalogs define graph identity; TS0 defines reusable statistics machinery .

The module provides:

- exact discrete probability mass functions for nonnegative integer counts;
- population scalar summaries;
- catalog-state occupancy and Shannon diversity;
- immutable frame/sample axes;
- immutable scalar series;
- state-level to frame-level expansion;

- schema-checked dictionary serialization;
- deterministic canonical JSON and SHA-256 payload digests.

Motivation

Atomic and framework statistics share mathematical operations even though they summarize different graph layers. For example, the following descriptors all use the same exact integer-distribution machinery:

- Na-O atomic contact count;
- projected framework-edge count;
- connected-component count;
- graph cycle-space rank;
- state residence length.

Implementing these operations separately in `atomic.py`, `framework.py`, and `temporal.py` would duplicate validation, quantile conventions, immutability, serialization, and error handling.

TS0 establishes one small common foundation so later modules can focus on their scientific responsibilities.

Responsibility boundaries

Owned by `_common.py`

The module owns:

- numeric input normalization;
- exact count support, frequency, and probability arrays;
- descriptive population statistics;
- catalog occupancy arrays and diversity metrics;
- trajectory visit-count bookkeeping;
- frame and sample alignment metadata;
- deterministic state-to-frame expansion;
- read-only defensive arrays;
- common serialization schemas and stable payload digests.

Not owned by `_common.py`

The module must not:

- inspect `AtomicEdgeKey` or `FrameworkEdgeKey`;
- count species pairs;
- calculate graph degree;
- classify framework bridge signatures;
- infer transitions from catalog edge differences;
- interpret ensemble order as time;
- calculate autocorrelation, lifetimes, or rates;
- create plots or write CSV files;
- rebuild connectivity or topology catalogs.

Module layout and import policy

The implementation lives in a private file:

```
mdstats/analysis/topology_statistics/_common.py
```

Users should import the supported names from:

```
from mdstats.analysis.topology_statistics import ...
```

The leading underscore indicates that later topology-statistics modules may use additional private helpers that are not part of the stable public API. The names explicitly re-exported by `topology_statistics/__init__.py` are public.

Public constants

```
CANONICAL_TOPOLOGY_STATISTICS_COMMON_SCHEMA = (  
    "mdstats.topology-statistics.common.v1"  
)  
TOPOLOGY_STATISTICS_DIGEST_ALGORITHM = "sha256"  
DEFAULT_QUANTILES = (0.0, 0.25, 0.5, 0.75, 1.0)
```

DEFAULT_QUANTILES represents minimum, lower quartile, median, upper quartile, and maximum.

Exception hierarchy

```
TopologyStatisticsError  
|-- TopologyStatisticsInputError  
|-- TopologyStatisticsConsistencyError  
`-- TopologyStatisticsSerializationError
```

- TopologyStatisticsInputError reports malformed user-facing numeric inputs, dimensions, quantiles, state IDs, or frame counts.
- TopologyStatisticsConsistencyError reports disagreement inside a constructed result object.
- TopologyStatisticsSerializationError reports incompatible schemas, object types, or noncanonical payloads.

All three derive from ValueError through TopologyStatisticsError.

Mathematical conventions

Population summary

For a nonempty scalar collection $x_1, \dots, x_F$, TS0 defines

$$\mu = \frac{1}{F} \sum_{f=1}^F x_f,$$

and the population standard deviation

$$\sigma = \sqrt{\frac{1}{F} \sum_{f=1}^F (x_f - \mu)^2}.$$

The implementation uses `ddof=0`. This is a descriptive summary of the analyzed collection, not an independent-sample uncertainty estimate.

Exact discrete distribution

For a nonnegative integer count series n_f , the exact support probability is

$$p(k) = \frac{1}{F} \sum_{f=1}^F \mathbf{1}[n_f = k].$$

The analysis stores exact support values, integer frequencies, and probabilities. It does not choose histogram bins.

Catalog occupancy

For dense catalog state ID s ,

$$n_s = \sum_{f=1}^F \mathbf{1}[s_f = s], \quad p_s = \frac{n_s}{F}.$$

The Shannon state entropy is

$$H = - \sum_{s: p_s > 0} p_s \ln p_s,$$

and the effective populated-state count is

$$N_{\text{eff}} = \exp(H).$$

This quantity is descriptive **Shannon state entropy**, not thermodynamic entropy.

Data structures

ScalarSummary

```
@dataclass(frozen=True, slots=True)
class ScalarSummary:
    count: int
    mean: float
    population_standard_deviation: float
    minimum: float
    maximum: float
    median: float
    quantile_probabilities: NDArray[np.float64]
    quantile_values: NDArray[np.float64]
    is_constant: bool
```

Invariants

- count ≥ 1 ;
- all scalar values are finite;
- standard deviation is nonnegative;
- minimum does not exceed median or maximum;
- quantile probabilities are strictly increasing from exactly 0 to exactly 1;
- quantile values are finite, aligned, and nondecreasing;
- first and last quantile values equal minimum and maximum;
- is_constant is true exactly when minimum equals maximum;
- a constant summary has exactly zero population standard deviation;
- arrays are copied and read-only.

Serialization

```
summary.to_dict()
ScalarSummary.from_dict(payload)
```

The payload contains the common schema and object type.

DiscreteCountDistribution

```
@dataclass(frozen=True, slots=True)
class DiscreteCountDistribution:
    support: NDArray[np.int64]
    frequencies: NDArray[np.int64]
    probabilities: NDArray[np.float64]
    summary: ScalarSummary
    modes: tuple[int, ...]
```

Meaning

For support values k_j ,

$$\text{frequencies}[j] = \#\{f : n_f = k_j\},$$

and

$$\text{probabilities}[j] = \frac{\text{frequencies}[j]}{F}.$$

All tied maximum-frequency support values appear in modes.

Properties and queries

```
n_observations
n_support_values
is_constant
frequency_for(count)
probability_for(count)
```

An unobserved count returns zero frequency and probability.

Invariants

- support is nonempty, strictly increasing, integer, and nonnegative;
- frequencies are positive integers;
- probabilities are finite, nonnegative, and exactly consistent with frequencies;
- the summary count, mean, and extrema agree with the PMF;
- modes are sorted, unique, and exactly the maximum-frequency support values;
- arrays are copied and read-only.

StateFrameGroup

```
@dataclass(frozen=True, slots=True)
class StateFrameGroup:
    state_id: int
    result_positions: NDArray[np.int64]
```

One object records all selected result positions assigned to one dense catalog state ID. The position array may be empty when a filtered analysis preserves an unobserved state from a parent catalog.

Properties:

```
frame_count
first_result_position
last_result_position
```

The first and last properties return None for an empty group.

CatalogOccupancyStatistics

```
@dataclass(frozen=True, slots=True)
class CatalogOccupancyStatistics:
    frame_semantics: FrameSemantics
    frame_to_state_id: NDArray[np.int64]
    state_frame_counts: NDArray[np.int64]
    state_probabilities: NDArray[np.float64]
    first_result_positions: NDArray[np.int64]
    last_result_positions: NDArray[np.int64]
    visit_counts: NDArray[np.int64] | None
    frame_groups: tuple[StateFrameGroup, ...]
    dominant_state_ids: tuple[int, ...]
    singleton_state_ids: tuple[int, ...]
    shannon_state_entropy: float
    effective_state_count: float
```

Dense state contract

State IDs are zero-based dense indices into the declared state arrays:

$$0 \leq s_f < N_s.$$

`n_states` may exceed the largest observed state ID. Unobserved declared states have:

- frame count zero;
- probability zero;
- first and last result positions -1;
- an empty `StateFrameGroup`;

- zero trajectory visits when trajectory semantics apply.

Trajectory versus ensemble behavior

For `FrameSemantics.TRAJECTORY`, `visit_counts[s]` is the number of maximal contiguous runs of state s .

For `FrameSemantics.ENSEMBLE`, `visit_counts` must be `None`. Stored ensemble order does not define visits.

Properties

`n_frames`
`n_states`
`n_observed_states`
`unobserved_state_ids`

`dominant_state_ids` contains all states tied for maximum frame count.

FrameAxis

```
@dataclass(frozen=True, slots=True)
class FrameAxis:
    frame_semantics: FrameSemantics
    collection_frame_indices: NDArray[np.int64]
    frame_ids: NDArray[np.int64]
    steps: NDArray[np.int64] | None = None
    times: NDArray[np.float64] | None = None
    time_unit: str | None = None
```

Meaning

- `collection_frame_indices` locates selected frames in the source collection;
- `frame_ids` preserves source frame identity;
- `steps` stores optional simulation-step values;
- `times` stores optional physical times;
- `time_unit` labels physical times.

Trajectory constraints

- frame indices are unique, nonnegative, and strictly increasing;
- optional steps align with frames and are strictly increasing;
- optional times are finite, aligned, and strictly increasing;
- a nonempty `time_unit` is required with times;
- a time unit without times is invalid.

Ensemble constraints

Ensemble axes reject steps, times, and `time_unit`. They expose sample order only.

Properties

`n_frames`
`result_positions`
`has_physical_time`
`x_values`
`x_label`

The display-axis preference is:

1. physical times;
2. simulation steps;
3. frame index for trajectories;
4. sample index for ensembles.

`x_values` is presentation metadata only. It does not authorize temporal statistics for ensembles.

ScalarSeries

```
@dataclass(frozen=True, slots=True)
class ScalarSeries:
    name: str
    values: NDArray[np.int64] | NDArray[np.float64]
    axis: FrameAxis
    summary: ScalarSummary
    unit: str | None = None
```

Invariants

- the name is nonempty;
- values are finite, one-dimensional, and aligned with the axis;
- integer inputs remain int64; floating inputs become float64;
- the stored summary exactly matches the values;
- an optional unit is nonempty;
- arrays are copied and read-only.

Property:

is_integer

Public functions

compute_scalar_summary

```
def compute_scalar_summary(
    values: ArrayLike,
    *,
    quantiles: Sequence[float] | ArrayLike = DEFAULT_QUANTILES,
) -> ScalarSummary:
    ...
```

Input constraints

- values is one-dimensional, numeric, finite, and nonempty;
- booleans are rejected;
- quantiles are finite and strictly increasing from 0 to 1 inclusive.

Algorithm

1. Convert integer input to int64 or floating input to float64.
2. Compute minimum, maximum, mean, median, and ddof=0 standard deviation.
3. Compute quantiles with NumPy's deterministic linear interpolation method.
4. Construct a validated immutable ScalarSummary.

Time complexity is $O(F \log F)$ in the worst case because quantile evaluation may require ordering work. Memory is $O(F)$ due to defensive copying.

compute_discrete_count_distribution

```
def compute_discrete_count_distribution(
    counts: ArrayLike,
    *,
    quantiles: Sequence[float] | ArrayLike = DEFAULT_QUANTILES,
) -> DiscreteCountDistribution:
    ...
```

Input constraints

- one-dimensional integer input;
- nonempty;
- nonnegative;
- booleans and floating arrays are rejected, even if a float happens to hold an integer value.

Algorithm

1. Compute sorted unique support and exact frequencies with `numpy.unique`.
2. Divide frequencies by observation count.
3. Identify all tied modes.
4. Compute a scalar population summary from the original count series.
5. Construct an immutable exact PMF.

Time complexity is $O(F \log F)$ under sorting-based uniqueness. Memory is $O(F)$.

`compute_catalog_occupancy_statistics`

```
def compute_catalog_occupancy_statistics(
    frame_to_state_id: ArrayLike,
    *,
    frame_semantics: FrameSemantics | str,
    n_states: int | None = None,
) -> CatalogOccupancyStatistics:
    ...
```

Input constraints

- assignments are one-dimensional nonnegative integers;
- at least one frame is required;
- supplied `n_states` is positive and exceeds every observed state ID;
- semantics must coerce to `TRAJECTORY` or `ENSEMBLE`.

Algorithm

1. Count state assignments using `numpy.bincount`.
2. Build exact probabilities.
3. Record first and last result positions and one `StateFrameGroup` per state.
4. For trajectories only, count starts of maximal contiguous state runs.
5. Compute dominant and singleton states.
6. Compute Shannon entropy over positive-probability states and exponentiate it.
7. Construct an immutable validated result.

Time complexity is $O(F + N_s)$. Memory is $O(F + N_s)$, including preserved frame assignments and frame groups.

`build_frame_axis`

```
def build_frame_axis(
    n_frames: int,
    *,
    frame_semantics: FrameSemantics | str,
    collection_frame_indices: ArrayLike | None = None,
    frame_ids: ArrayLike | None = None,
    steps: ArrayLike | None = None,
    times: ArrayLike | None = None,
    time_unit: str | None = None,
) -> FrameAxis:
    ...
```

Default collection indices are `0, ..., n_frames-1`. Default frame IDs equal the collection indices.

The function validates lengths before constructing `FrameAxis`.

`build_scalar_series`

```
def build_scalar_series(
    name: str,
    values: ArrayLike,
    axis: FrameAxis,
    *,
    unit: str | None = None,
```

```

    quantiles: Sequence[float] | ArrayLike = DEFAULT_QUANTILES,
) -> ScalarSeries:
    ...

```

The function computes the summary and constructs an aligned immutable series.

expand_state_values_to_frames

```

def expand_state_values_to_frames(
    state_values: ArrayLike,
    frame_to_state_id: ArrayLike,
) -> NDArray[Any]:
    ...

```

If state-level data have shape

$$(N_s, d_1, \dots, d_m),$$

the result has shape

$$(F, d_1, \dots, d_m).$$

The leading axis is always the dense state axis. Trailing dimensions and dtype are preserved. The returned array is copied and read-only.

Time and memory complexity are proportional to the expanded output size.

Canonical JSON and payload digest

```

def canonical_statistics_json(payload: Mapping[str, Any]) -> str:
    ...

def topology_statistics_payload_digest(payload: Mapping[str, Any]) -> str:
    ...

```

Canonical JSON:

- recursively converts NumPy arrays and scalar values;
- converts FrameSemantics to its string value;
- sorts mapping keys;
- uses compact separators;
- rejects nonfinite JSON numbers;
- does not depend on Python mapping insertion order.

The digest is

$$D = \text{SHA256}(\text{UTF8}(\text{canonical JSON})).$$

These helpers provide deterministic building blocks for later result schemas. They do not define graph identity.

Serialization contract

Every TS0 result `to_dict()` payload contains:

```

{
    "schema_version": "mdstats.topology-statistics.common.v1",
    "object_type": "...",
}

```

`from_dict()` must reject:

- another schema version;
- another object type;
- malformed arrays or inconsistent summaries;
- unsupported scalar-series dtypes;

- semantics-inconsistent temporal metadata.

Deserialization re-runs all constructor invariants. Serialized values cannot bypass validation.

Immutability and defensive copying

All numeric arrays are:

1. copied from caller-owned input;
2. normalized to `int64` or `float64` where specified;
3. marked read-only.

A caller changing the original input array after construction must not alter the result. Attempts to write into result arrays should fail.

Dataclasses are frozen and slotted. Downstream modules must create new results rather than mutate shared common objects.

Trajectory and ensemble semantics

TS0 preserves but does not invent semantic meaning.

Operation	Trajectory	Ensemble
Exact count distribution	yes	yes
State occupancy	yes	yes
Shannon diversity	yes	yes
First/last stored result position	yes	yes, indexing only
Disjoint visit count	yes	no
Simulation steps	optional	rejected
Physical time	optional	rejected
Sample index	indexing only	indexing only

First and last positions in an ensemble are storage positions, not temporal bounds.

Edge cases and warnings

Constant series

A constant count distribution is valid and important. For 2,000 frames each with 96 Si-O edges,

$$p(96) = 1, \quad \sigma = 0.$$

TS0 must not treat zero width as an error.

Multiple modes

Discrete distributions may have several tied modes. All are retained in sorted order.

Unobserved declared states

A filtered selection may preserve the parent catalog's full state count. Zero- occupancy states remain represented but do not contribute to entropy.

Correlated trajectory frames

The reported standard deviation is descriptive. It is not a standard error and must not be interpreted as one.

Large state expansions

`expand_state_values_to_frames()` materializes its full output. A later module should avoid expanding large high-dimensional state descriptors unless needed.

Floating-point quantiles

Quantiles use linear interpolation and may be noninteger even when the original series contains integer counts. Exact frequencies remain authoritative.

Time duration

FrameAxis stores sampled time coordinates only. Residence-duration conventions belong to `temporal.py` and are intentionally not defined in TS0.

Usage examples

Exact constant contact distribution

```
import numpy as np
from mdstats import compute_discrete_count_distribution

si_o = compute_discrete_count_distribution(
    np.full(2000, 96, dtype=np.int64)
)

assert si_o.support.tolist() == [96]
assert si_o.frequencies.tolist() == [2000]
assert si_o.proBABILITIES.tolist() == [1.0]
```

Recurrent trajectory-state occupancy

```
from mdstats import FrameSemantics, compute_catalog_occupancy_statistics

stats = compute_catalog_occupancy_statistics(
    [0, 0, 1, 1, 0],
    frame_semantics=FrameSemantics.TRAJECTORY,
)

assert stats.state_frame_counts.tolist() == [3, 2]
assert stats.visit_counts.tolist() == [2, 1]
```

Immutable time-aligned count series

```
from mdstats import FrameSemantics, build_frame_axis, build_scalar_series

axis = build_frame_axis(
    3,
    frame_semantics=FrameSemantics.TRAJECTORY,
    times=[0.0, 0.001, 0.002],
    time_unit="ps",
)

series = build_scalar_series(
    "Na-0 contacts",
    [114, 116, 115],
    axis,
    unit="edges",
)
```

Testing requirements

The TS0 focused suite must cover:

- population rather than sample standard deviation;
- default and custom quantiles;
- constant scalar summaries;
- exact delta-function count distributions;
- tied modes;

- rejection of empty, negative, float, Boolean, and nonfinite count inputs;
- trajectory occupancy and recurrent visits;
- ensemble occupancy without visits;
- unobserved declared states;
- zero entropy for one occupied state;
- axis precedence: time, step, then frame/sample index;
- ensemble rejection of temporal metadata;
- monotonic trajectory frame, step, and time validation;
- scalar-series alignment and immutability;
- state-to-frame expansion with trailing dimensions;
- round-trip serialization;
- schema and object-type rejection;
- order-independent canonical JSON and digests;
- public package re-exports.

Implementation notes for later stages

TS1 atomic statistics - implemented

`atomic.py` reuses:

- `DiscreteCountDistribution` for species-pair counts;
- `CatalogOccupancyStatistics` for atomic-state occupancy;
- `ScalarSeries` for per-frame total and pair counts;
- `expand_state_values_to_frames()` for catalog-compressed descriptors.

It does not add atomic meaning to `_common.py`. Cross-state occupancy and event identity use gauge-invariant atom pairs in TS1, not common-layer records.

TS2 framework statistics - implemented

`framework.py` reuses the same primitives for projected edge counts, components, degree summaries, and cycle-space rank. Whole-path bridge signatures remain framework-specific.

TS3 temporal statistics - implemented

`temporal.py` consumes `FrameAxis`, but owns residence intervals, transition matrices, return lags, entity episodes, and duration conventions. These responsibilities remain outside TS0.

Accepted TS0 decisions

1. `_common.py` is graph-independent.
2. Exact integer PMFs are authoritative for count descriptors.
3. Scalar standard deviation uses `ddof=0`.
4. Quantiles include exact 0 and 1 endpoints.
5. Common result arrays are defensive and read-only.
6. Dense state IDs are zero-based array indices.
7. Unobserved declared states may be retained explicitly.
8. Shannon entropy excludes zero-probability terms.
9. Visit counts exist only for trajectories.
10. Ensemble axes reject physical time and simulation steps.
11. `FrameAxis` does not define residence durations.
12. State expansion preserves trailing shape and dtype.
13. Serialization is schema- and object-type checked.
14. Stable payload digests summarize serialized statistics data, not graph identity.
15. Plotting and file export remain outside TS0.

Final TS0 contract

The TS0 transformation is

```
numeric descriptor values or dense catalog assignments
      |
      v
validated immutable common statistics objects
      |
      +--> TS1 atomic statistics
      +--> TS2 framework statistics
      +--> TS3 temporal statistics
      `--> TS4 combined statistics
```

The implementation is complete only when its scientific meaning remains generic:

`_common.py` knows statistics and frame semantics, but not graph chemistry.